

Task 1: Capabilities of CampusEats

1. **Manage profile and addresses:** Allows students to log in, maintain their user profiles, and save/manage multiple delivery addresses.
2. **Browse restaurants and menus:** Enables users to explore various campus restaurants, view their available food items, and check current prices.
3. **Add to cart and place order:** Allows students to select items, add them to a shopping cart, and finalize their food order.
4. **Pay online and process refunds:** Handles the financial transactions, charging the user for their placed order and managing refunds if necessary.
5. **Assign rider and track delivery:** Responsible for allocating an available delivery rider to an order and providing real-time tracking updates to the student.
6. **Send notifications:** Dispatches automated alerts to the student at key order stages (e.g., order placed, payment successful, on the way, delivered).

Task 3: Service Contracts

1. Orders Service (Main Service)

This service acts as the central orchestrator.

Operation	Input	Output	Errors
addToCart	studentId, itemId, qty	cart	item unavailable
placeOrder	studentId, cart, addressId	orderId, status, total	empty cart, bad address
getOrder	orderId	order + status	not found
cancelOrder	orderId	status: cancelled	too late to cancel

2. Catalogue Service (Main Service)

This service manages all food items and restaurant data.

Operation	Input	Output	Errors
listRestaurants	area / filter	restaurants[]	—
getMenu	restaurantId	menu items + prices	not found
checkItem	itemId	available?, price	not found

3. Payments Service

Handles all financial transactions securely.

Operation	Input	Output	Errors
charge	amount, method	receipt	payment declined
refund	txnId	status	invalid txnId

4. Delivery Service

Manages riders and logistics.

Operation	Input	Output	Errors
assignRider	orderId, addressId	rider details, ETA	no riders available

5. Notifications Service

Handles all system-to-user communication.

Operation	Input	Output	Errors
send	studentId, message	status: sent	invalid user

6. Accounts Service

Manages user profiles and addresses.

Operation	Input	Output	Errors
getAddress	studentId, addressId	address details	address not found

Task 4: Full Specification of Central Operation (placeOrder)

Operation Name: *placeOrder*

Inputs (IN):

- studentId
- items: [{itemId, qty}]
- deliveryAddressId
- paymentMethodId

Outputs (OUT):

- orderId
- status: 'placed'
- total
- estimatedMinutes

Error Cases (ERRORS):

- item unavailable
- payment declined
- invalid address
- empty cart

Internal Details Hidden From Callers:

- The orders database table
- How the internal IDs are generated
- Which specific delivery rider gets picked
- The specific payment gateway being used

Task 6: Service Validation Table

Below is the validation of the CampusEats microservices architecture against the five core properties of a web service. Because the design strictly adheres to data ownership boundaries, all services pass successfully.

Property	Validation Check for CampusEats Services	Result
Reachable over a network	All services (Orders, Catalogue, Payments, etc.) expose published operations (e.g., placeOrder, charge, checkItem) that other services can call over the network.	PASS
Self-contained	Every service executes its complete job independently. For instance, Payments does the whole payment job itself and exclusively owns the Transactions data.	PASS
Has a contract	Every service has a clearly defined contract specifying operations, inputs, outputs, and explicit error cases (as defined in Task 3) without exposing SQL or table names.	PASS
Independent	Services can change their internal schema without breaking callers. For example, Catalogue can rebuild its database overnight, and Orders will not notice as long as the contract holds.	PASS
Loosely coupled	Services never read each other's database tables. Orders does not read the Menus table; it only calls the checkItem operation and trusts the reply.	PASS

Note on Failures & Fixes:

During the initial conceptualization, there was a risk of a boundary failure if the Orders service directly queried the Users table (owned by Accounts) to fetch delivery addresses.

- **Why it would fail:** This would break the **Independent** and **Loosely Coupled** properties because two services would share the same table. Neither could change the schema without breaking the other.
- **How it was fixed:** We enforced the rule of "one table, one owner". The Orders service now merely stores the student_id and address_id, and if it needs address details, it must call the Accounts service's contract (getAddress) instead of touching its database directly.